

VULNERABILITY ASSESSMENT DELIVERABLE

OWASP Top 10 & Vulnerability Register

Full OWASP Top 10 Matrix & Detailed Technical Findings with Remediation Proof

Coverage: 10/10 OWASP Top 10 (2021) Categories **Total Findings:** 10 Cataloged Findings (4 Critical, 1 High)

Remediation: 6 Remediated & Retested via Burp Suite

Risk Management: 4 Documented Accepted Risks & Future Work

Part 1: OWASP Top 10 Assessment Matrix

A rigorous evaluation of every category in the current OWASP Top 10 was performed across the MediConnect platform using a combination of automated static analysis (Semgrep), dependency scanning (`npm audit`), secret detection (Gitleaks), and manual dynamic testing (Burp Suite Community Edition).

#	CATEGORY	TESTS PERFORMED	RESULT	TECHNICAL EVIDENCE	FINDING IDS	SEVERITY	STATUS
A01	Broken Access Control	IDOR testing on chat history, prescription termination, unread metadata; vertical privilege escalation via registration; route-level role middleware audit	Finding	Burp Repeater before/after validation	AS-01, AS-03, AS-04, AS-05, AS-06	Critical	Remediated
A02	Cryptographic Failures	JWT algorithm verification (HS256), bcrypt password hashing (10 salt rounds), TLS assessment, secret configuration analysis	Finding	Semgrep SG-02 (TLS verification bypass in local dev config)	SG-02	Low	Accepted Risk
A03	Injection	SQL injection audit on all 40 endpoints, XSS testing via input fields, command injection review	Pass	All database queries use parameterized <code>\$1, \$2</code> syntax; Zod schema input validation	—	—	Pass
A04	Insecure Design	Business logic review, AI authorization architecture, brute-force resistance	Finding	No rate limiting on login endpoint; AI access unconstrained prior to hardening	AS-08	Medium	Partially Fixed

#	CATEGORY	TESTS PERFORMED	RESULT	TECHNICAL EVIDENCE	FINDING IDS	SEVERITY	STATUS
A05	Security Misconfiguration	Helmet header validation, CORS policy audit, debug routes, default credentials	Finding	Missing CSRF token middleware; CORS development configuration	SG-01	Medium	Accepted Risk
A06	Vulnerable & Outdated Components	Software Composition Analysis (<code>npm audit</code>), Semgrep dependency rules	Finding	Known CVEs in transitive dependencies (<code>jws < 3.2.3</code> , <code>engine.io</code> , <code>socket.io-parser</code>)	DEP-01	Low-High	Future Work
A07	Identification & Authentication Failures	Session invalidation, token revocation on logout, brute-force credential stuffing	Finding	No server-side token blacklist upon logout; brute-force login vulnerability	AS-07, AS-08	Medium	Partially Fixed
A08	Software & Data Integrity Failures	CI/CD pipeline integrity, dependency lockfile verification, integrity checksums	Pass	<code>package-lock.json</code> cryptographic integrity; automated CI/CD pipeline	—	—	Pass
A09	Security Logging & Monitoring Failures	Security audit logging, authentication event capture, real-time alert generation	Finding	Absence of security event logging and detection controls (prior to hardening)	—	Medium	Remediated
A10	Server-Side Request Forgery (SSRF)	Outbound request patterns, AI API integrations,	N/A	Application makes outbound HTTP requests solely to Google	—	—	N/A (Justified)

#	CATEGORY	TESTS PERFORMED	RESULT	TECHNICAL EVIDENCE	FINDING IDS	SEVERITY	STATUS
		external webhooks		Gemini API via hardcoded endpoint; no user-controllable URLs			

OWASP Assessment Summary

- **Categories with Findings:** 7
- **Categories Passed:** 2
- **Categories Not Applicable:** 1 (with architectural justification)
- **Total Evaluated:** 10 / 10 (100% coverage)

Part 2: Vulnerability Register

Summary of Vulnerabilities

SEVERITY	TOTAL IDENTIFIED	REMEDIATED & RETESTED	DOCUMENTED ACCEPTED RISK / FUTURE WORK
Critical	4	4	0
High	1	1	0
Medium	3	1	2
Low	2	0	2
Total	10	6	4

Critical Findings

AS-01 — Chat History IDOR (ePHI Exposure)

FIELD	DESCRIPTION
Finding ID	AS-01
Title	Insecure Direct Object Reference in Chat History API
Affected Component	<code>GET /api/chat/history/room/:roomId</code> — <code>chatController.js</code>
Severity / Likelihood	Critical / High
Security Impact	Complete unauthorized read access to private doctor-patient medical conversations
Business / Regulatory Impact	HIPAA Breach (§164.402) — unauthorized disclosure of patient symptoms and SSNs
Preconditions	Authenticated user with valid JWT (any role)
OWASP Category	A01:2021 — Broken Access Control
HIPAA Control	§164.312(a)(1) — Technical Access Control

- **Root Cause:** The endpoint accepted any `roomId` parameter (`${idA}-${idB}`) from the URL without verifying that the requesting user was a registered participant in that conversation.

- **Before Remediation (Evidence):** Patient B (`userId=2`) issued `GET /api/chat/history/room/1-3` . The server returned `200 OK` exposing Patient A's confidential medical message:
`"Doctor, I have severe chest pains and my SSN is 123-45-6789"`
- **Remediation:** Added server-side ownership validation in `chatController.js` :

```
const [idA, idB] = roomId.split('-').map(Number);
if (userId !== idA && userId !== idB) {
  return res.status(403).json({ message: 'Access denied: you are not a participant in this chat' });
}
```

- **Retest Result:** Retest via Burp Suite returns `403 Forbidden` (`{"message":"Access denied..."}`).
- **Status: Remediated & Verified**

AS-03 — Prescription Termination IDOR (Patient Safety)

FIELD	DESCRIPTION
Finding ID	AS-03
Title	Unauthorized Prescription Termination via Missing Ownership Check
Affected Component	<code>PUT /api/prescriptions/end/:id</code> — <code>prescriptionController.js</code>
Severity / Likelihood	Critical / High
Security Impact	Unauthorized modification and termination of active medical prescriptions
Business / Regulatory Impact	Patient safety risk — premature cancellation of critical patient medication; HIPAA §164.312(c)(1) violation
Preconditions	Authenticated user (any role)
OWASP Category	A01:2021 — Broken Access Control
HIPAA Control	§164.312(c)(1) — Data Integrity

- **Root Cause:** The update SQL query executed `UPDATE prescriptions SET status = 'ended' WHERE id = $1` without checking which doctor prescribed it or whether the requester was authorized.
- **Before Remediation (Evidence):** Patient B sent `PUT /api/prescriptions/end/1` targeting Patient A's active Amoxicillin prescription. The server responded `200 OK` (`"Prescription marked as ended"`).
- **Remediation:** Enforced doctor role middleware and added `doctor_id = $3` verification to the SQL query:

```
const result = await db.query(
  'UPDATE prescriptions SET status = $1 WHERE id = $2 AND doctor_id = $3',
  ['ended', id, doctorId]
);
```

- **Retest Result:** Retest returns `403 Forbidden`. Unauthorized users cannot alter any prescription.
- **Status: Remediated & Verified**

AS-04 — Unread Message Metadata IDOR

FIELD	DESCRIPTION
Finding ID	AS-04
Title	Unread Message Count Exposes Other Users' Clinical Communication Metadata
Affected Component	<code>GET /api/chat/unread?userId=X</code> — <code>chatController.js</code>
Severity / Likelihood	Critical / High
Security Impact	Leaks private user communication patterns and active conversation IDs
OWASP Category	A01:2021 — Broken Access Control
HIPAA Control	§164.312(a)(1) — Technical Access Control

- **Root Cause:** The controller extracted `userId` from user-supplied query parameters (`req.query.userId`) instead of trusting the cryptographically verified JWT (`req.user.userId`).
- **Before Remediation (Evidence):** Patient B queried `GET /api/chat/unread?userId=3` and received `{"1-3": 1}`, exposing the Doctor's unread message count and room ID.
- **Remediation:** Replaced query parameter with JWT claim:

```
const userId = req.user.userId;
```

- **Retest Result:** Endpoint returns `{}` (only Patient B's own unread count).
- **Status: Remediated & Verified**

AS-06 — Admin Self-Registration (Privilege Escalation)

FIELD	DESCRIPTION
Finding ID	AS-06
Title	Arbitrary Role Registration Allows Anyone to Become System Administrator
Affected Component	<code>POST /api/auth/register</code> — <code>authController.js</code>
Severity / Likelihood	Critical / High
Security Impact	Complete compromise of application; unrestricted access to all patients, doctors, and ePHI
Business / Regulatory Impact	Total administrative compromise; reportable HIPAA breach
OWASP Category	A01:2021 — Broken Access Control
HIPAA Control	§164.308(a)(4) — Information Access Management

- **Root Cause:** The registration endpoint validated roles against `['patient', 'doctor', 'admin']`, allowing any unauthenticated user to register directly as an admin.
- **Before Remediation (Evidence):** Sent registration payload with `{"role": "admin"}`. Server returned `201 Created` with admin JWT. Accessing `GET /api/admin/patients` returned full database records.
- **Remediation:** Removed `'admin'` from `validRoles`. Admin accounts can only be created directly in the database by infrastructure administrators:

```
const validRoles = ['patient', 'doctor'];
```

- **Retest Result:** Attempting to register with `admin` returns `400 Bad Request` (`{"message": "Invalid role"}`).
- **Status: Remediated & Verified**

High Findings

AS-05 — Missing Role Middleware on Prescription Endpoints

FIELD	DESCRIPTION
Finding ID	AS-05
Title	Prescription Routes Lack Role Authorization Middleware
Affected Component	backend/src/routes/prescriptionRoutes.js
Severity	High
Remediation	Applied authorizeRoles('doctor') and authorizeRoles('patient') across all routes.
Status:	Remediated & Verified

Medium Findings

- **AS-07 — No Token Revocation Mechanism (Medium):** Logout clears the client cookie, but signed JWTs remain valid until expiration (24h). *Status:* Accepted Risk (mitigated by HttpOnly, SameSite cookies). Future mitigation: Redis token blacklist.
- **AS-08 — No Rate Limiting on Authentication (Medium):** No automated request throttling on login. *Status:* Partially Mitigated via real-time brute-force detection alert in securityLogger.js.
- **SG-01 — Missing CSRF Protection Middleware (Medium):** Identified via Semgrep. *Status:* Accepted Risk (mitigated by SameSite=Lax cookie attribute and same-origin architecture).

Low Findings

- **SG-02 — TLS Verification Bypass in Database Config (Low):** rejectUnauthorized: false in db/index.js for development PostgreSQL. *Status:* Accepted Risk for local development environment. Production environment requires strict TLS.
- **DEP-01 — Vulnerable Dependencies in npm Packages (Low-High):** 92 transitive dependency vulnerabilities flagged by npm audit. *Status:* Future Work (planned major version upgrades).